

ZYNQ 程序固化教程

1、程序功能：

以 PL 侧按键控制 LED 灯为例，每次按下按键，对应的 LED 灯就亮。

2、源程序：

```
module key_led(  
    input  wire      clk,  
    input  wire      rst_n,  
    input  wire      [2:0]key,  
    output wire      [2:0]led  
);  
  
reg      [5:0]key_dd;  
always@(posedge clk or negedge rst_n)begin  
    if(rst_n==1'b0)begin  
        key_dd<='d0;  
    end  
    else begin  
        key_dd <= {key_dd[2:0],key};  
    end  
end  
  
assign led = key_dd[5:3];  
  
endmodule
```

3、管脚约束文件：

```
set_property IOSTANDARD LVCMOS33 [get_ports clk]  
set_property IOSTANDARD LVCMOS33 [get_ports rst_n]  
set_property IOSTANDARD LVCMOS33 [get_ports {key[2]}]
```

```

set_property IOSTANDARD LVCMOS33 [get_ports {key[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {key[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[0]}]
set_property PACKAGE_PIN H16 [get_ports clk]
set_property PACKAGE_PIN G14 [get_ports {led[0]}]
set_property PACKAGE_PIN C20 [get_ports {led[1]}]
set_property PACKAGE_PIN B20 [get_ports {led[2]}]
set_property PACKAGE_PIN R19 [get_ports {key[0]}]
set_property PACKAGE_PIN T19 [get_ports {key[1]}]
set_property PACKAGE_PIN J15 [get_ports {key[2]}]
set_property PACKAGE_PIN B19 [get_ports rst_n]

```

4、如图 1.2.8 所示，点击左侧导航栏（Flow Navigator）中的 IP Integrator 选项下的 Create Block Design。然后在弹出的对话框中输入所创建的 Block Design 的名称，在 Design name 栏中输入“ledBD”，也可以不输入，使用默认名“design_1”，点击“OK”。

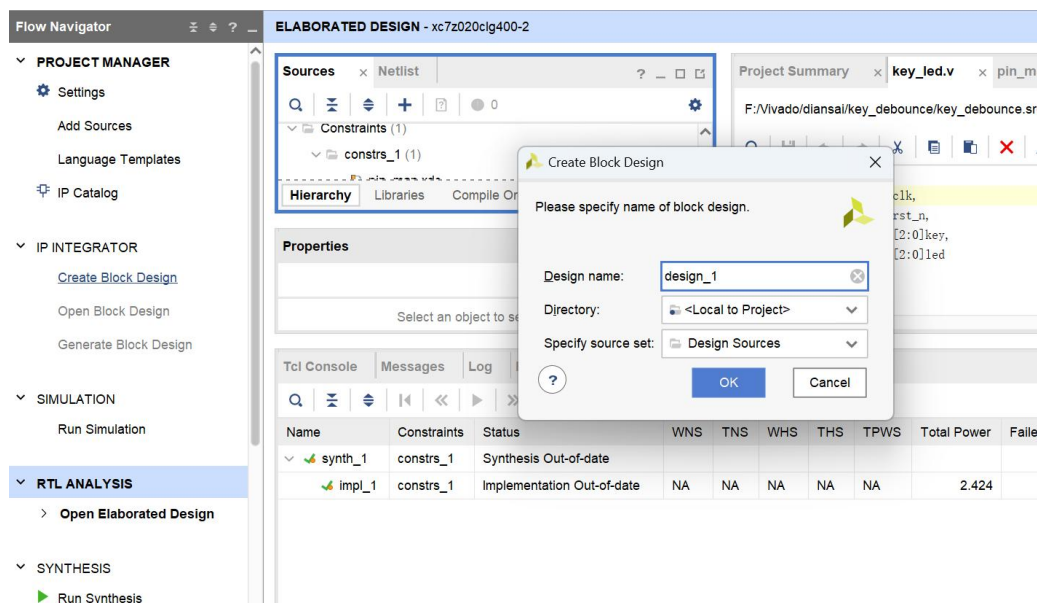


图 1.2.8 创建 BlockDesign

之后弹出 **Diagram** 窗口，如图 1.2.9 所示，我们将在此窗口下完成完成本实验硬件方面的设计。点击界面中间的“+”或者按快捷键 **Ctrl+I**，或者在空白区域点击鼠标右键，选择“**ADD IP**”。之后会弹出可添加的 **IP** 目录，在搜索栏中输入“**ZYNQ**”，在搜索结果中找到“**ZYNQ Processing System**”并双击，将该处理器系统 **IP** 添加到设计中。

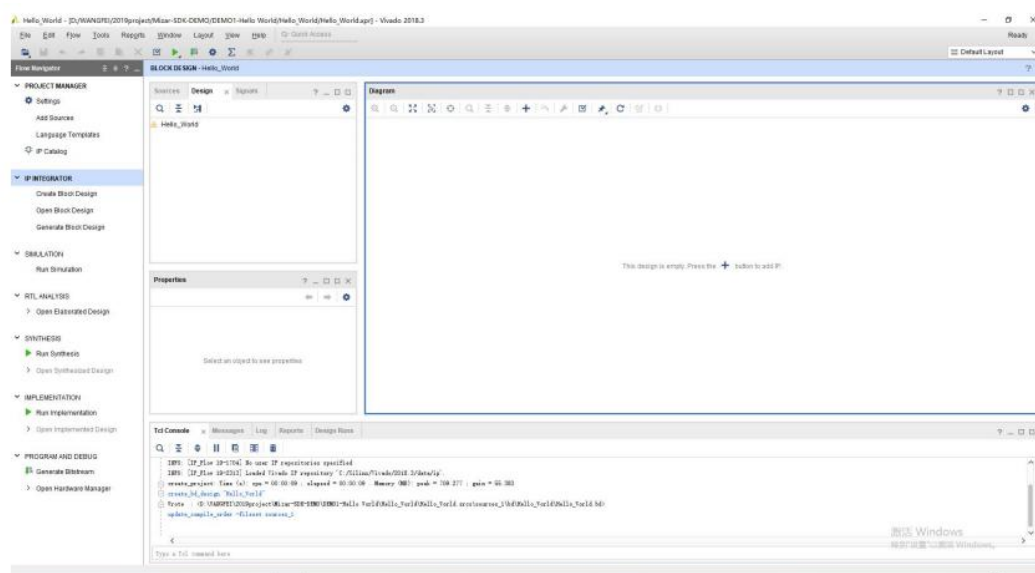


图 1.2.9 Block Design 界面

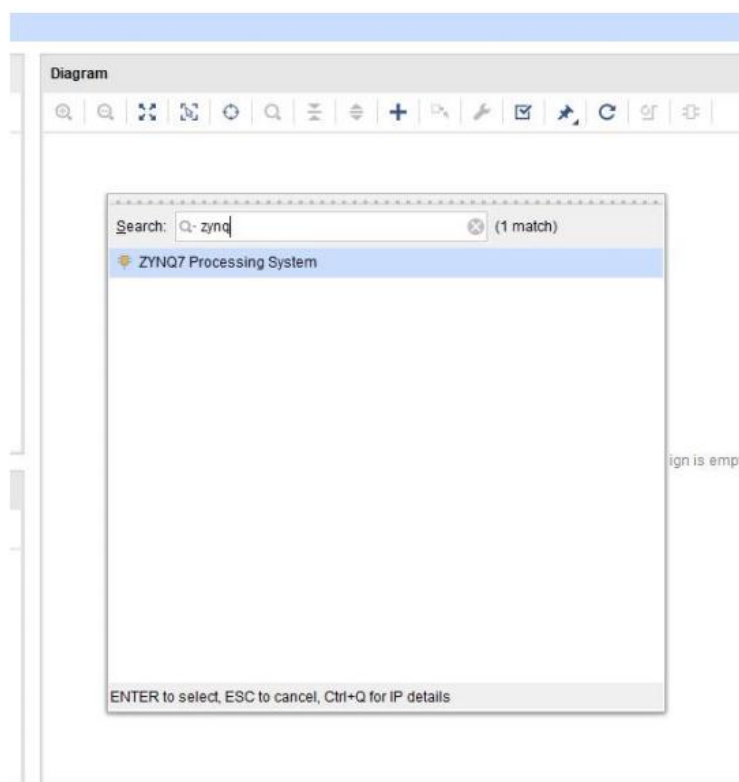


图 1.2.10 添加 ZYNQ 处理系统 IP

添加完成后，模块 IP 出现在 Diagram 中，如下图所示：

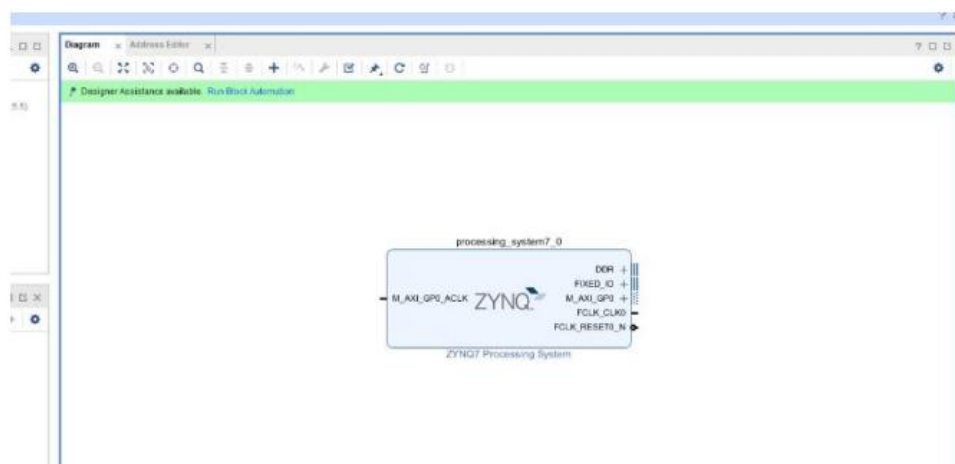


图 1.2.11 ZYNQ7 Processing System IP

双击添加的 ZYNQ7 Processing System 模块，进入 ZYNQ 处理系统配置界面。页面左侧显示导航面板，右侧为配置信息面板。如下图所示：

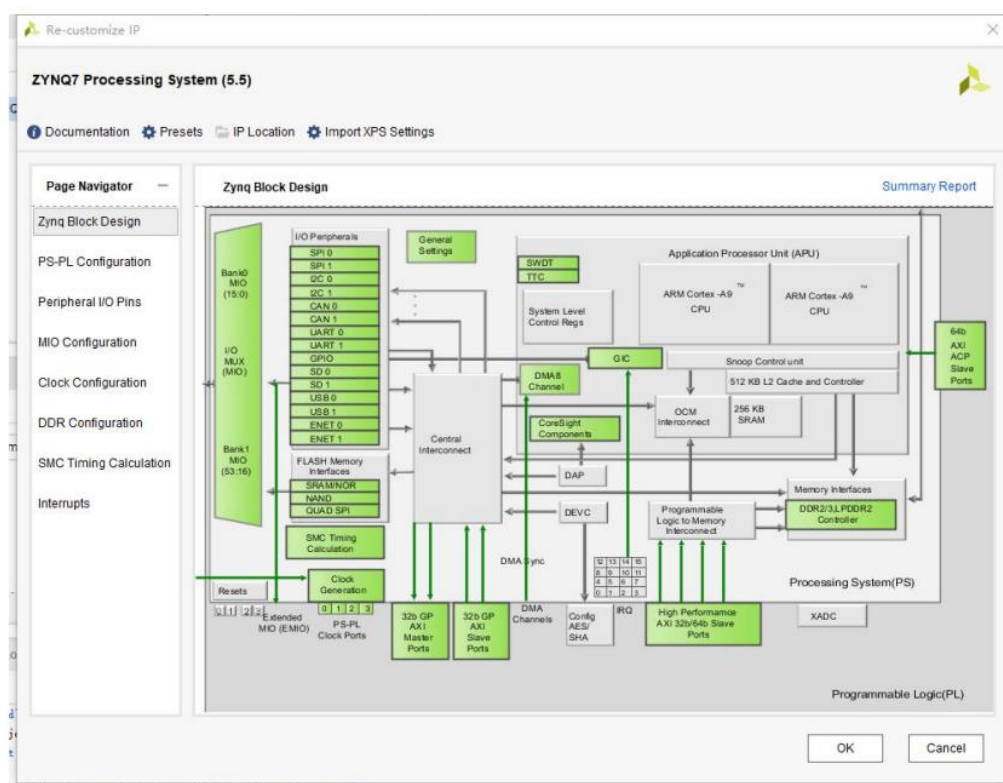


图 1.2.12 ZYNQ7 Processing System 配置界面

在该页面点击“Peripheral I/O Pins”，右侧显示出引脚配置界面，在这里我们可以看到哪些引脚可以被配置成那些相应的功能。本次实验我们需要用到串口的功能，但是图上有那么多引脚可以被配置为串口，我们应该怎么做呢？这里我们要通过查看原理图来确定哪些引脚应该被配置为串口的功能。如图 1.2.13 所

示，在开发板上 MIO48,MIO49 连接到串口芯片上实现串口的功能，所以我们要在 Vivado 软件上将 MIO48,MIO49 配置为串口。在配置界面点击 MIO48,MIO49 下面的 UART1,方块的颜色会变成绿色，同时两个 MIO 也变成绿色，表示这两个 I/O 已经和芯片内部的串口电路相连接,可以实现串口的功能,如图 1.2.14 所示。对于其他型号开发板，串口使用的 MIO 引脚可能不同，请自行查阅原理图进行设置，在后续的实验中不再进行特别提醒。

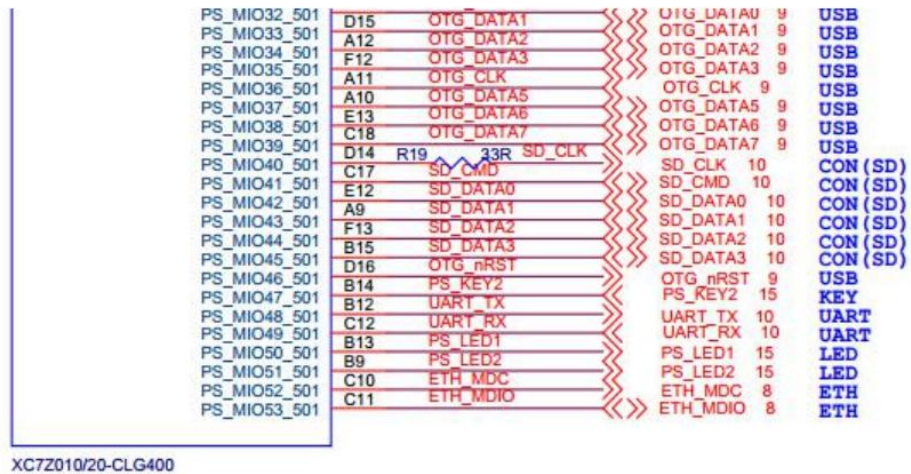
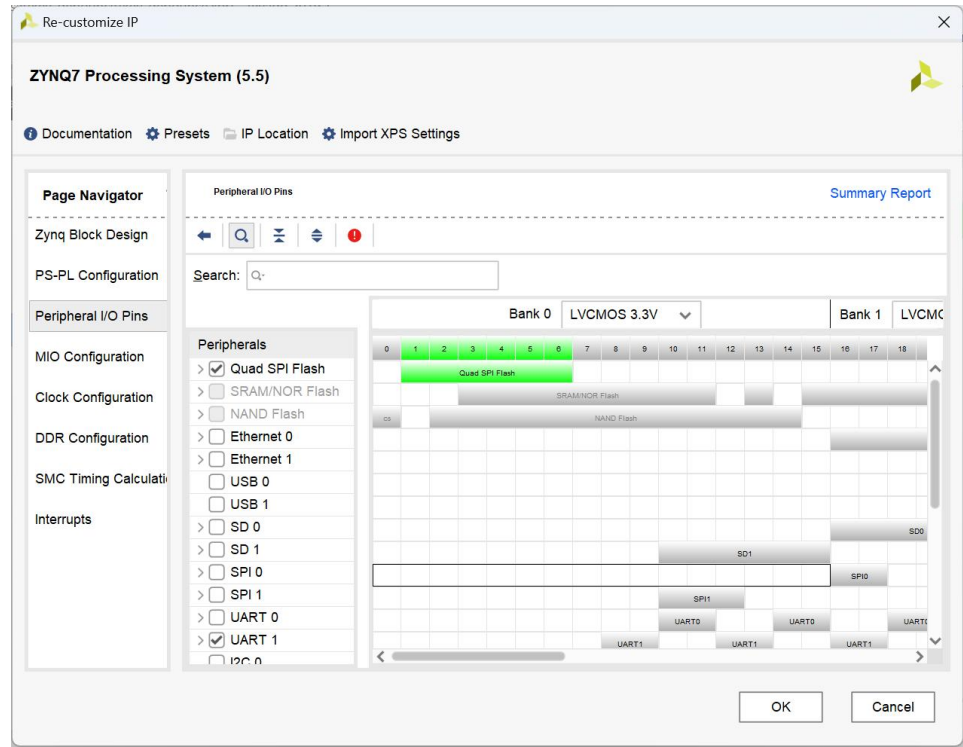


图 1.2.13 串口相关原理图

点亮下面 Quad SPI Flash，还有 UART1。



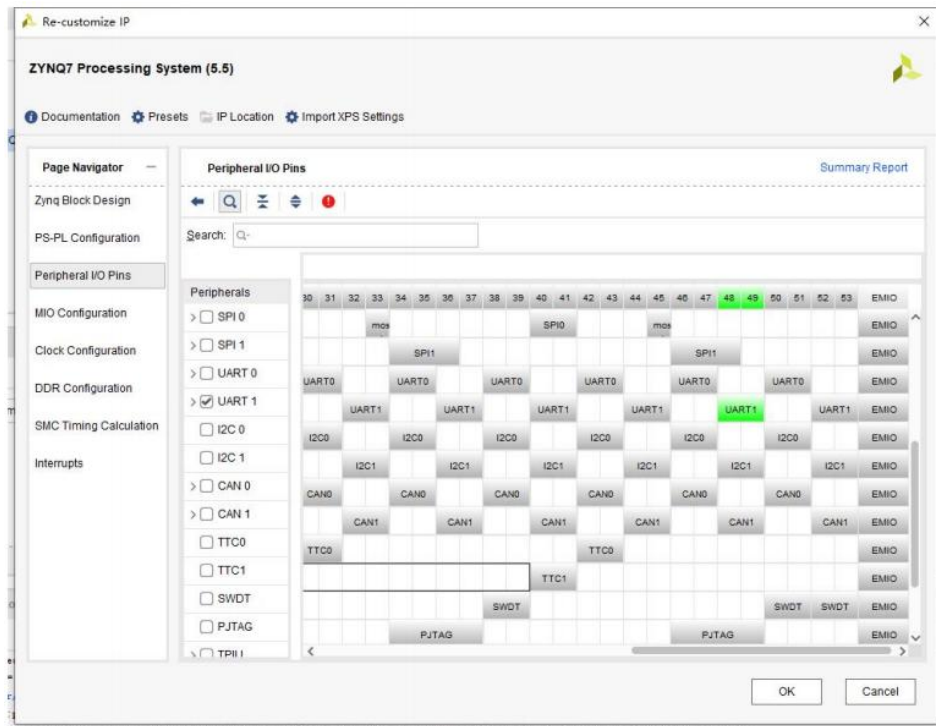
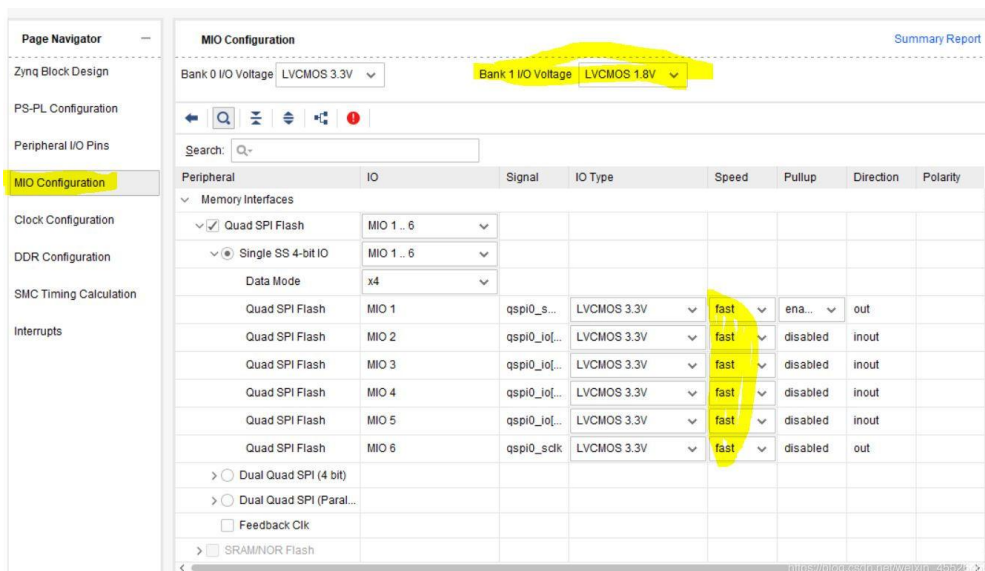


图 1.2.14 配置 MIO48 MIO49 为 UART1

点击左侧的“MIO Configuration”，MIO Configuration 选项中修改 Quad SPI Flash speed 为 fast。



在右侧展开 I/O Peripherals→UART1，因为我们使用的引脚为 MIO48 和 MIO49，属于 Bank1，所以此处将 Bank1 的引脚电平修改为 1.8V。在此处可以看到具体的引脚配置信息。其中 MIO48 为 TX 引脚方向为输入，MIO49 为 RX 引脚方向为输出，引脚电压为 1.8V，速度为 slow，使能上拉，如下图所示：

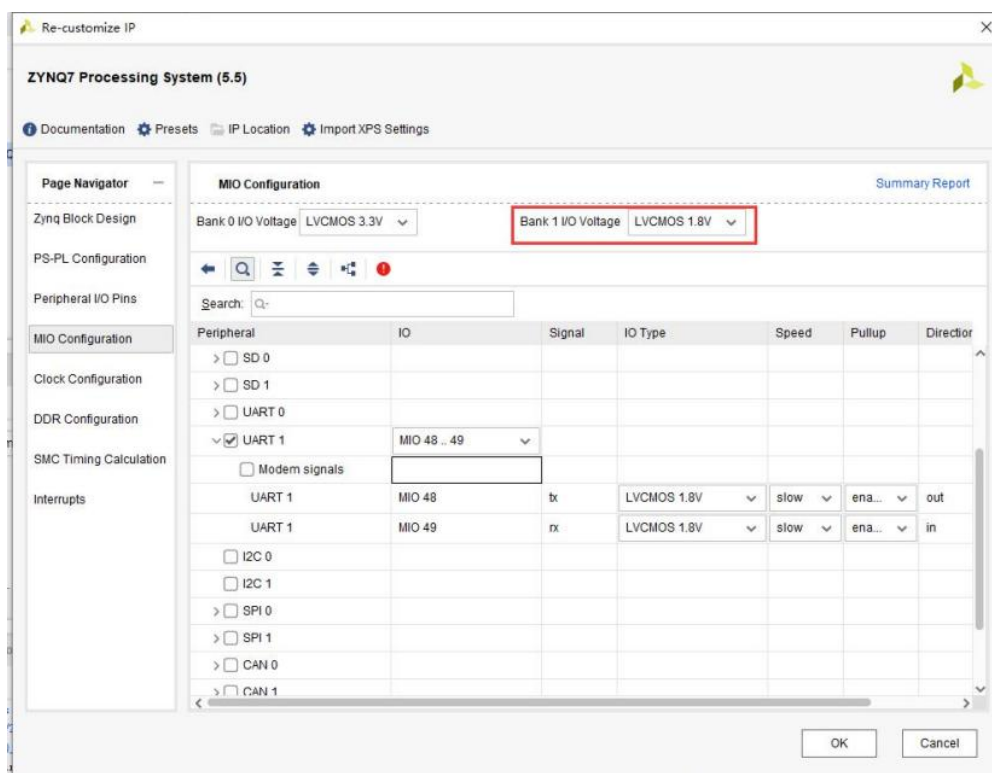


图 1.2.15 UART1 串口配置信息

除此之外，我们还要配置串口波特率等相关配置，点击左侧“PS-PL Configuration”，我们可以在这里设置波特率等设置，点开“General”可以看到默认的串口波特率为 115200，可以通过下拉按钮选择其他波特率，这里我们保持默认设置。

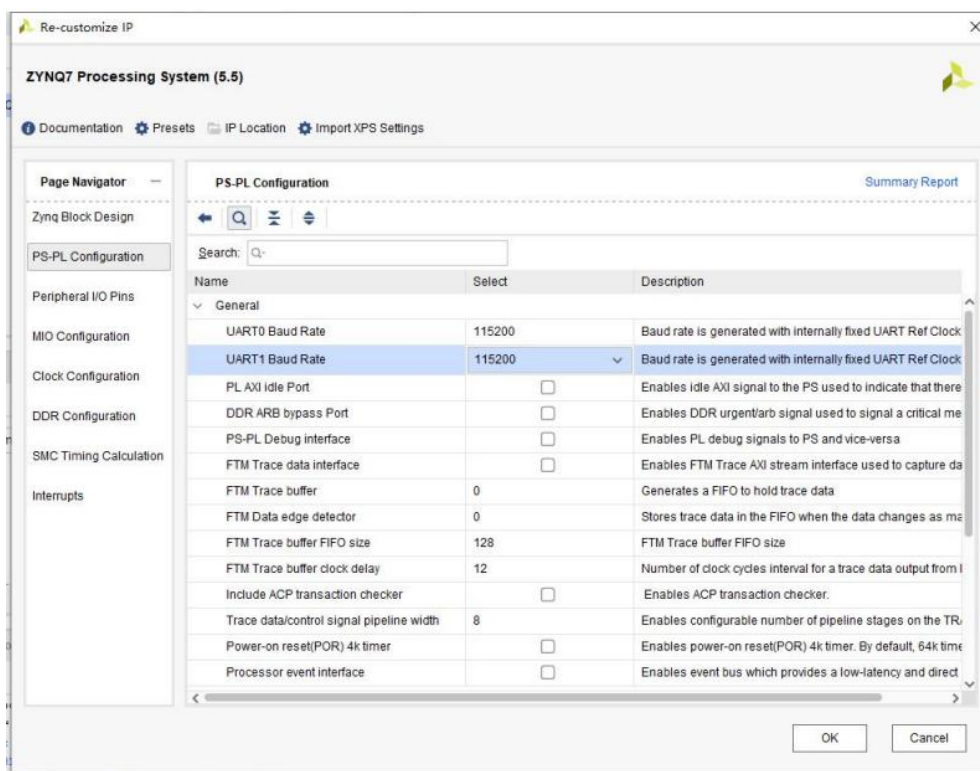


图 1.2.16 UART1 波特率设置

点击左侧“DDR Configuration”，选择 DDR 器件的型号，点开 DDR Controller Configuration 下拉按钮，在 Memory Part 一栏根据自己开发板的型号选择。芯片为 XC7Z010 的开发板选择 MT41J128M16 HA-125。芯片为其他型号的开发板全部选择“MT41J256M16 RE-125”。

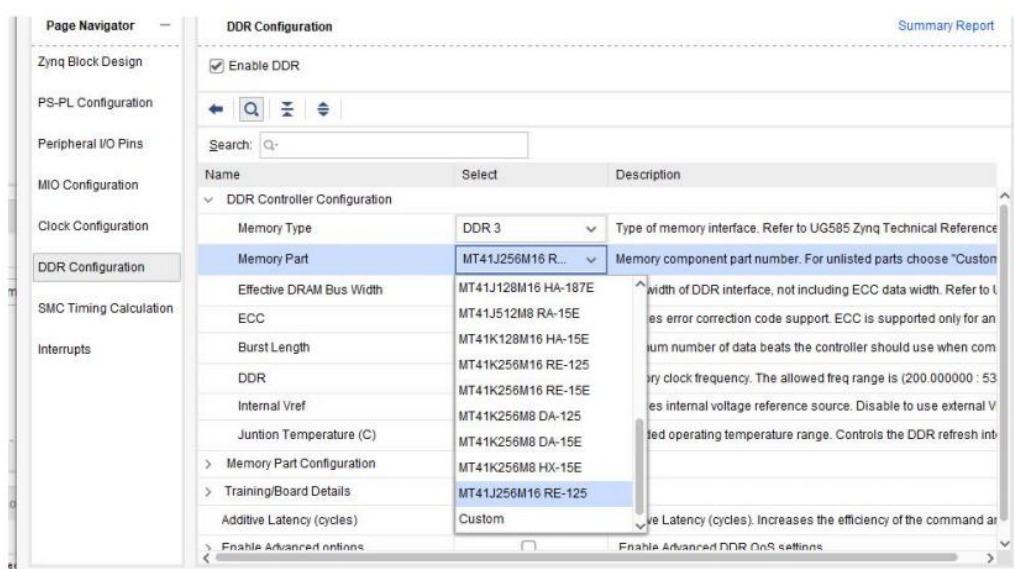


图 1.2.17 配置 DDR3 控制器

点击左侧“Clock Configuration”，显示 ZYNQ PS 端的时钟频率配置页面，此处的各个时钟频率保持默认设置，因为本实验不需要用到 PL 部分，所以点开

“PL Fabric Clock”，取消勾选“FCLK_CLK0”，如图 1.2.18 所示。

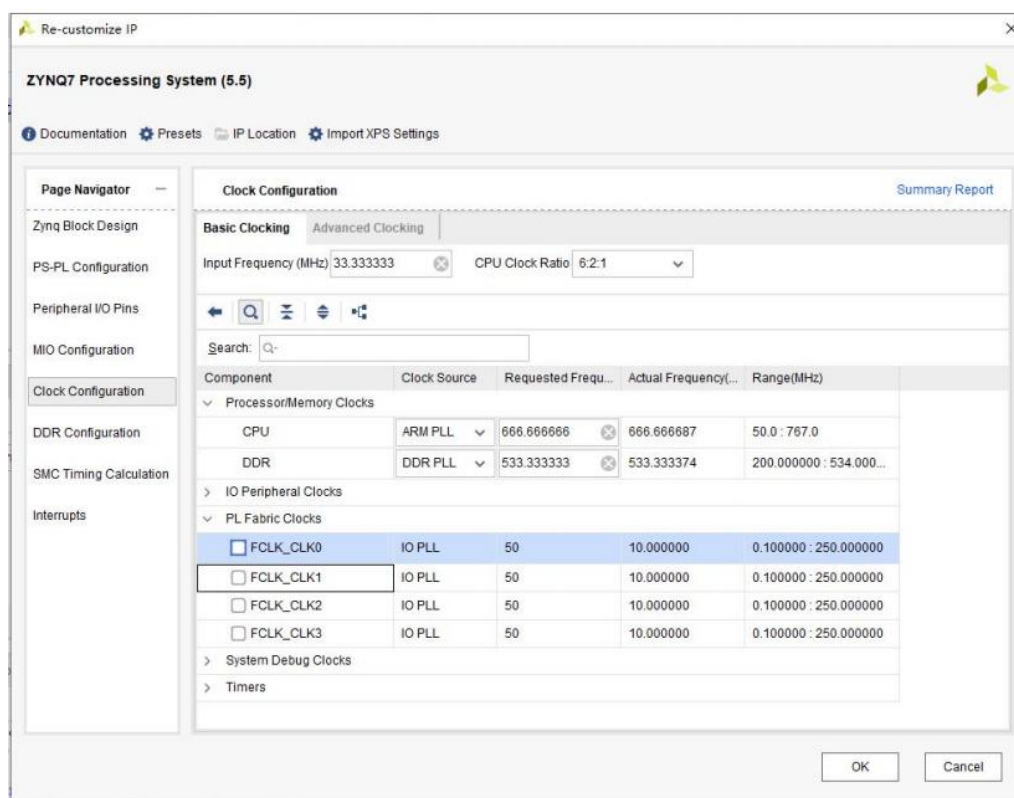


图 1.2.18 配置相关时钟

除此之外，我们还要取消一些与 PL 相关设置的勾选，点击“PS-PL Configuration”，在右侧展开 General 下的 Enable Clock Resets，取消勾选其中的 FCLK_RESETO_N。

另外在当前界面中展开 AXI Non Secure Enablement 下的 GP Master AXI Interface，取消勾选其中的 MAXI GP0 interface。如下图所示：

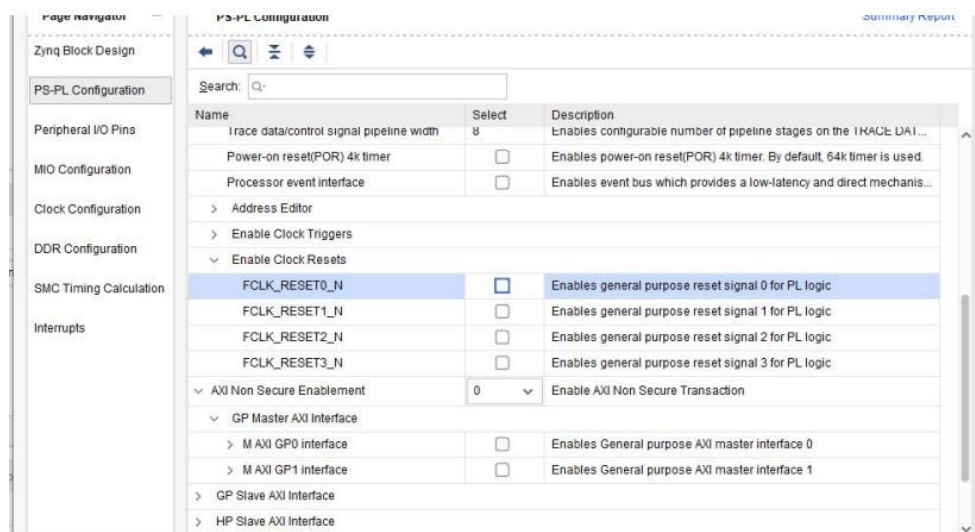


图 1.2.19 取消勾选 PL 相关设置

这样我们的配置就完成了，点击“OK”，保存配置并返回。

回到主界面我们会看到 ZYNQ7 模块发生了变化，如图 1.2.20，对比之前发现少了部分接口，这是因为我们取消了关于 PL 的部分设置。

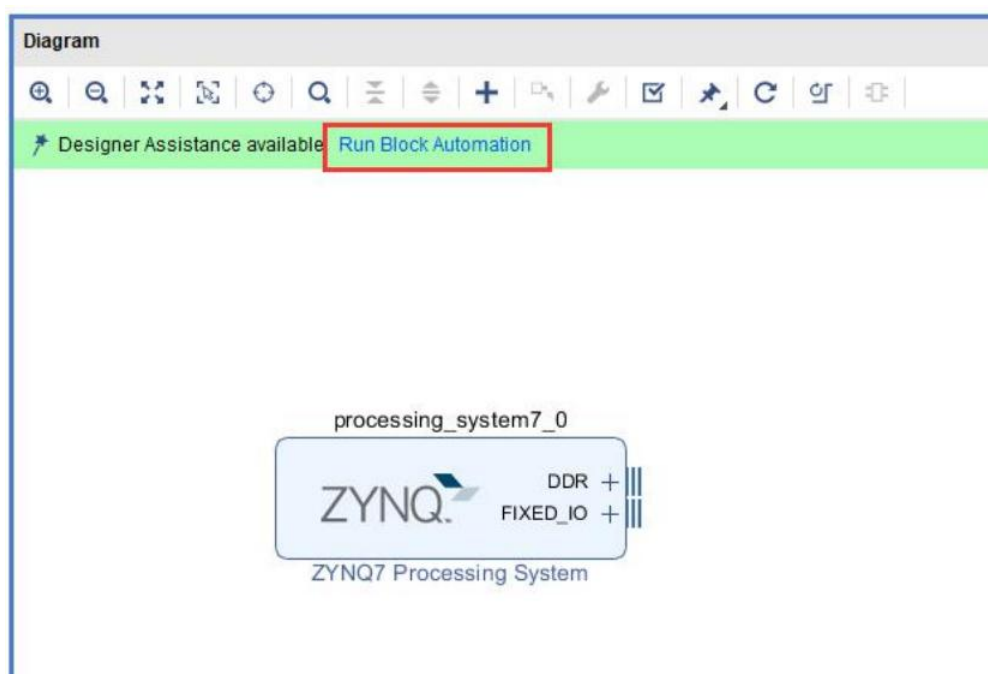


图 1.2.20 配置完成

之后点击上图中的“Run Block Automation”，会弹出如下对话框：

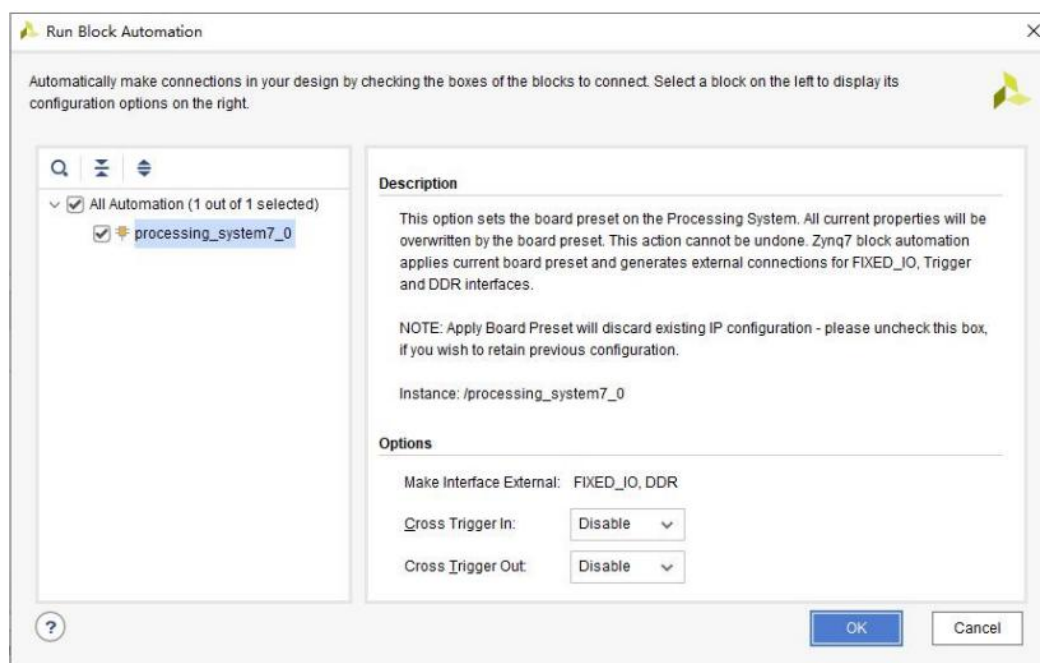


图 1.2.21 自动配置端口及 IP 间的互联

全选，点击“OK”，软件会自动为 IP 添加输入输出端口以及各个 IP 之间的连接，因为本实验只用到一个 IP，所以系统只引出了两组信号。

配置完成的 IP 如下图所示，点击接口旁边的“+”可以看到展开两组接口，查看其中具体的信号有哪些。

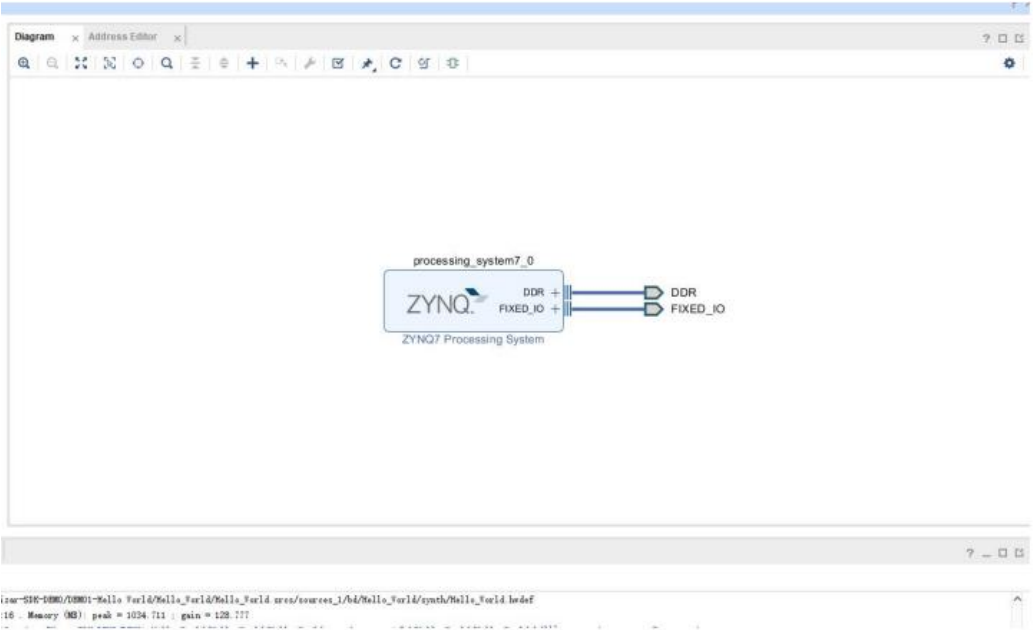


图 1.2.22 配置完成的 IP

当前实验不需要添加其他 IP，快捷键 **Ctrl + S** 保存当前设计，点击工具栏的 **Validate Design** 按钮验证当前的设计。验证完成后弹出对话框提示没有错误或者关键警告，点击“OK”，如下图所示：

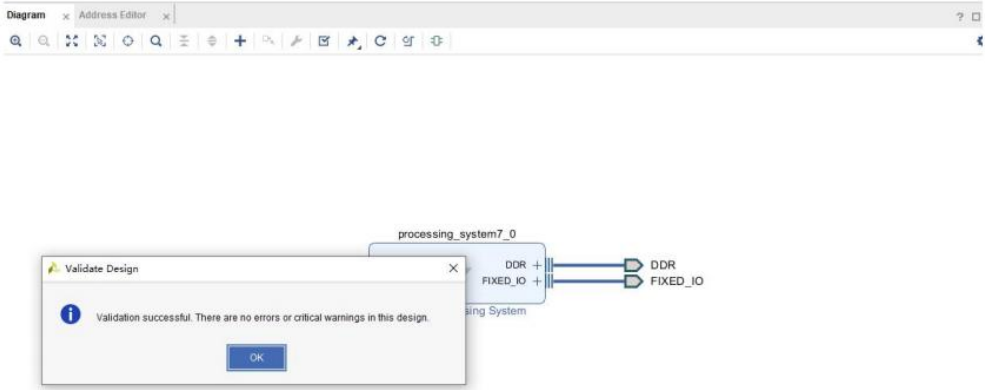


图 1.2.23 验证设计

如果出现错误或严重警告说明之前的配置有问题，需要返回重新检查配置是否有遗漏或错误。

设计完成后需要生成顶层 HDL 模块，在 Sources 窗口，选中 Design Source 下的 design_1.bd，右键点击 design_1.bd，在弹出的菜单中选择“Generate Output Products”，如下图所示：

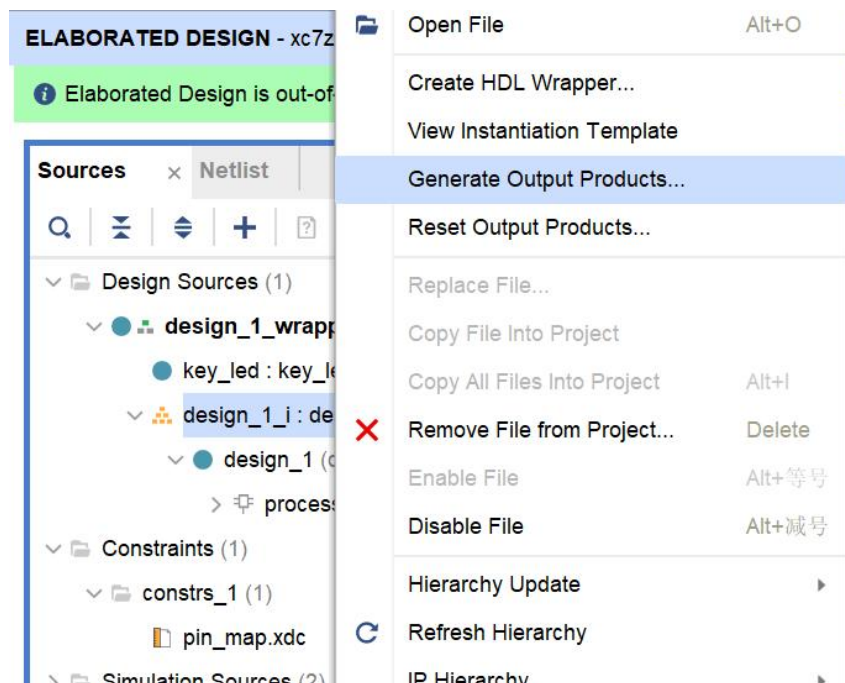


图 1.2.24 选择 Generate Output Products

弹出“Generate Output Products”对话框，如下图所示：

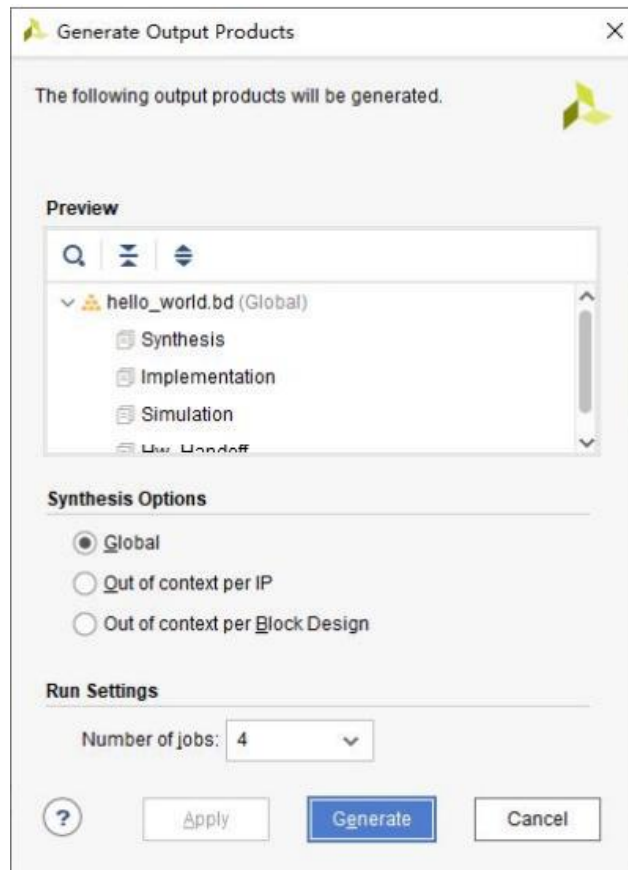


图 1.2.25 设置 Generate 选项

在对话框中 Synthesis Options 选择 Global；Run Settings 用于设置生成过程中要使用的处理器的线程数，进行多线程处理，保持默认或设置为个人电脑最大可使用线程数的一半。然后点击“Generate”来生成设计的综合、实现和仿真文件。

Generate 完成后，在弹出的对话框中点击“OK”。

在 Sources 窗口，选中 Design Source 下的 design_1.bd，右键点击 design_1.bd，在弹出的菜单中选择“Create HDL Wrapper”，如下图所示：

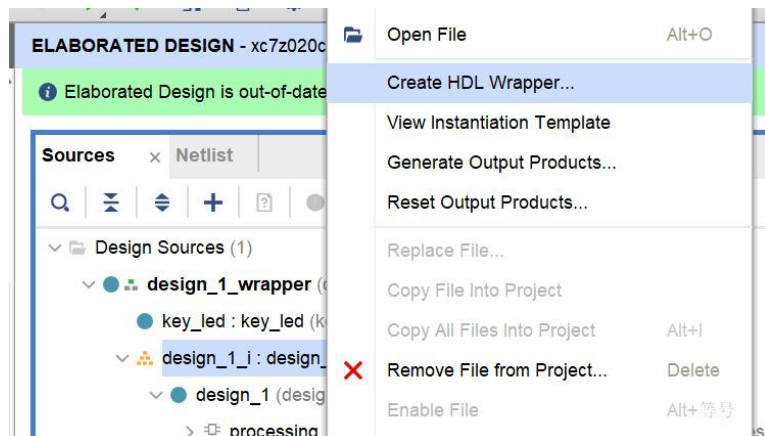


图 1.2.26 选择 Create HDL Wrapper

在弹出的对话框中确认勾选“Let Vivado manage wrapper and auto-update”，然后点击“OK”。

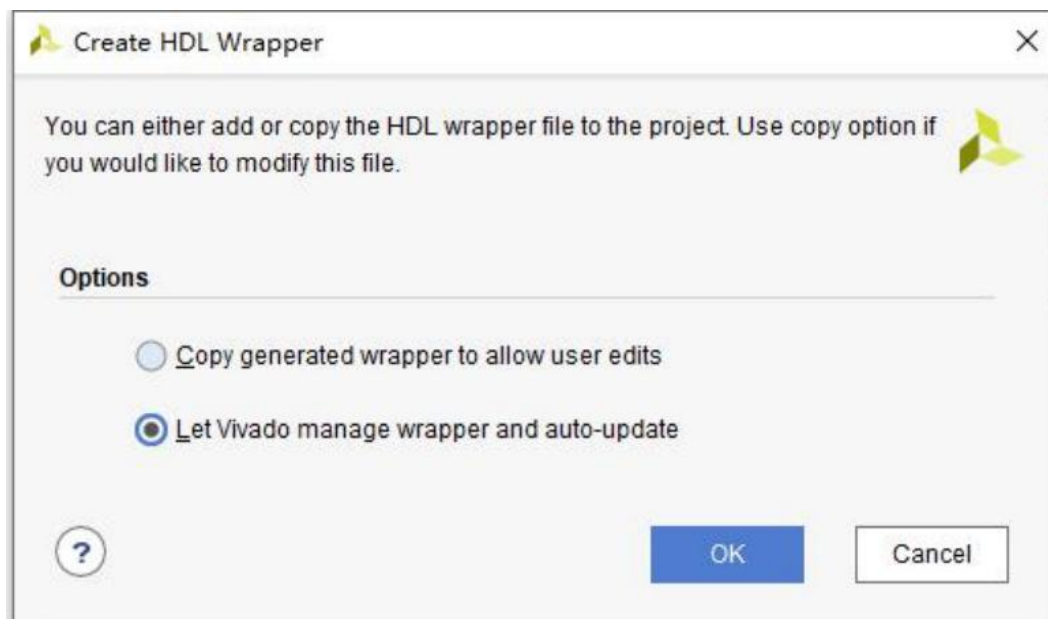


图 1.2.27 建立顶层 HDL

这样我们的硬件设计就创建完成了，完成的 Design Source 结构如下图所示：

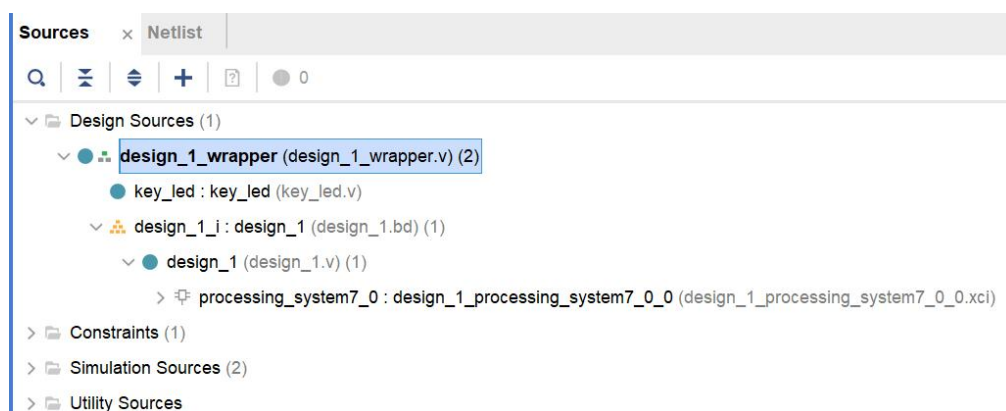


图 1.2.28 工程结构

以上部分在 ZYNQ 开发教程里 SDK 篇有详细记录，下面的步骤为本人在网上整理得。

例化：

双击打开 design_1wrapper.v 文件，把 key_led.v 写进 design_1wrapper.v，修改后的代码（粗体处是新加的 key_led.v 部分，其他地方默认不用改，关于例化的含义此处不再说明，此处已默认读者会例化子模块）：

```
module design_1_wrapper
```



```

(DDR_addr,
  DDR_ba,
  DDR_cas_n,
  DDR_ck_n,
  DDR_ck_p,
  DDR_cke,
  DDR_cs_n,
  DDR_dm,
  DDR_dq,
  DDR_dqs_n,
  DDR_dqs_p,
  DDR_odt,
  DDR_ras_n,
  DDR_reset_n,
  DDR_we_n,
  FIXED_IO_dds_vrn,
  FIXED_IO_dds_vrp,
  FIXED_IO_mio,
  FIXED_IO_ps_clk,
  FIXED_IO_ps_porb,
  FIXED_IO_ps_srstb,
  clk,
  rst_n,
  key,
  led);
input clk;
input rst_n;
input [2:0]key;
output [2:0]led;
inout [14:0]DDR_addr;

```

```
inout [2:0]DDR_ba;

inout DDR_cas_n;

inout DDR_ck_n;

inout DDR_ck_p;

inout DDR_cke;

inout DDR_cs_n;

inout [3:0]DDR_dm;

inout [31:0]DDR_dq;

inout [3:0]DDR_dqs_n;

inout [3:0]DDR_dqs_p;

inout DDR_odt;

inout DDR_ras_n;

inout DDR_reset_n;

inout DDR_we_n;

inout FIXED_IO_dds_vrn;

inout FIXED_IO_dds_vrp;

inout [53:0]FIXED_IO_mio;

inout FIXED_IO_ps_clk;

inout FIXED_IO_ps_porb;

inout FIXED_IO_ps_srstb;
```

```
wire [14:0]DDR_addr;

wire [2:0]DDR_ba;

wire DDR_cas_n;

wire DDR_ck_n;

wire DDR_ck_p;

wire DDR_cke;

wire DDR_cs_n;

wire [3:0]DDR_dm;
```

```

wire [31:0]DDR_dq;

wire [3:0]DDR_dqs_n;

wire [3:0]DDR_dqs_p;

wire DDR_odt;

wire DDR_ras_n;

wire DDR_reset_n;

wire DDR_we_n;

wire FIXED_IO_ddr_vrn;

wire FIXED_IO_ddr_vrp;

wire [53:0]FIXED_IO_mio;

wire FIXED_IO_ps_clk;

wire FIXED_IO_ps_porb;

wire FIXED_IO_ps_srstb;

wire clk;

wire rst_n;

wire [2:0]key;

wire [2:0]led;


key_led key_led(

.clk(clk),

.rst_n(rst_n),

.key(key),

.led(led)

);


design_1 design_1_i

    (.DDR_addr(DDR_addr),

    .DDR_ba(DDR_ba),

    .DDR_cas_n(DDR_cas_n),

    .DDR_ck_n(DDR_ck_n),

```

```

.DDR_ck_p(DDR_ck_p),

.DDR_cke(DDR_cke),

.DDR_cs_n(DDR_cs_n),

.DDR_dm(DDR_dm),

.DDR_dq(DDR_dq),

.DDR_dqs_n(DDR_dqs_n),

.DDR_dqs_p(DDR_dqs_p),

.DDR_odt(DDR_odt),

.DDR_ras_n(DDR_ras_n),

.DDR_reset_n(DDR_reset_n),

.DDR_we_n(DDR_we_n),

.FIXED_IO_ddr_vrn(FIXED_IO_ddr_vrn),

.FIXED_IO_ddr_vrp(FIXED_IO_ddr_vrp),

.FIXED_IO_mio(FIXED_IO_mio),

.FIXED_IO_ps_clk(FIXED_IO_ps_clk),

.FIXED_IO_ps_porb(FIXED_IO_ps_porb),

.FIXED_IO_ps_srstb(FIXED_IO_ps_srstb));

```

endmodule

重新编译一下工程，生成 bit 文件。

转入 SDK

1. Launch SDK

在菜单栏选择 File > Export > Export hardware。

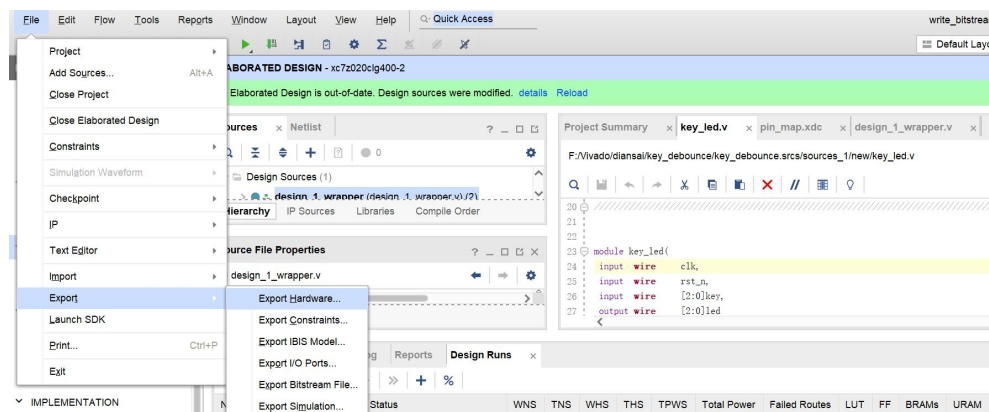


图 1.2.29 导出硬件到 SDK

在弹出的对话框中可以选则是否包含 Bitstream 文件，因为我们生成了 Bitstream 文件，选择包含 bitstram 文件。

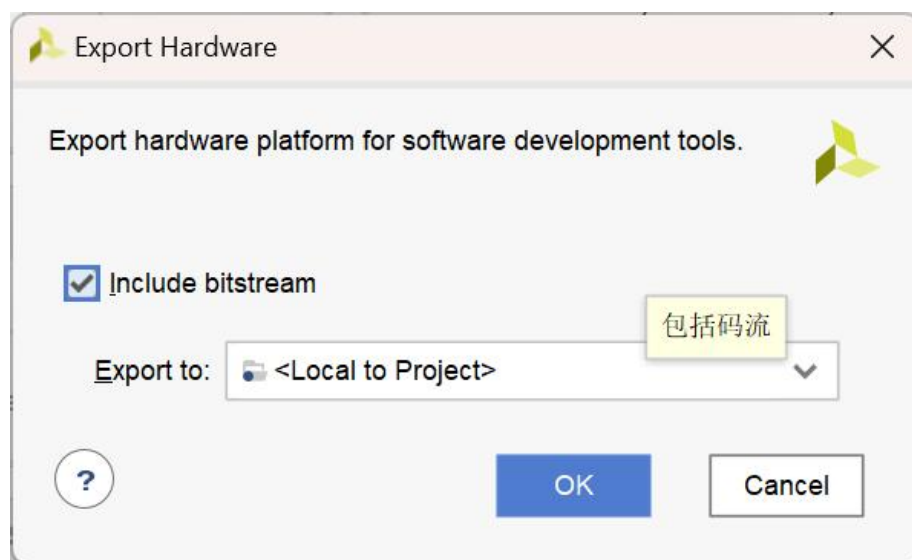


图 1.2.30 导出选项

硬件导出完成后，在菜单栏中选择 File > Launch SDK，启动 SDK 开发环境。
如下图所示：

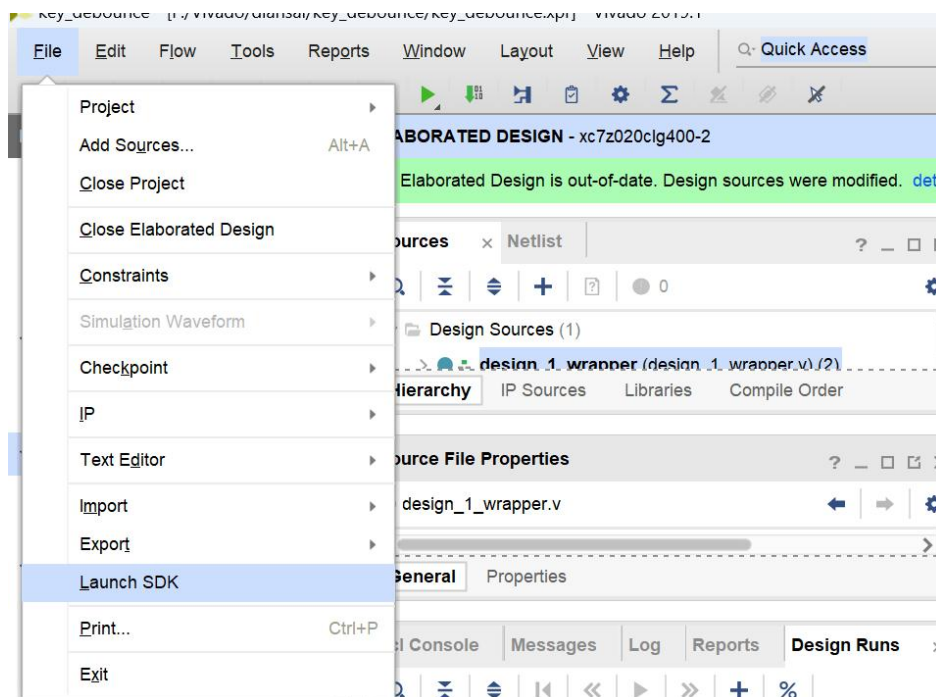


图 1.2.31 启动 SDK 开发环境

在弹出的对话框中，直接点击“OK”，如下图所示：

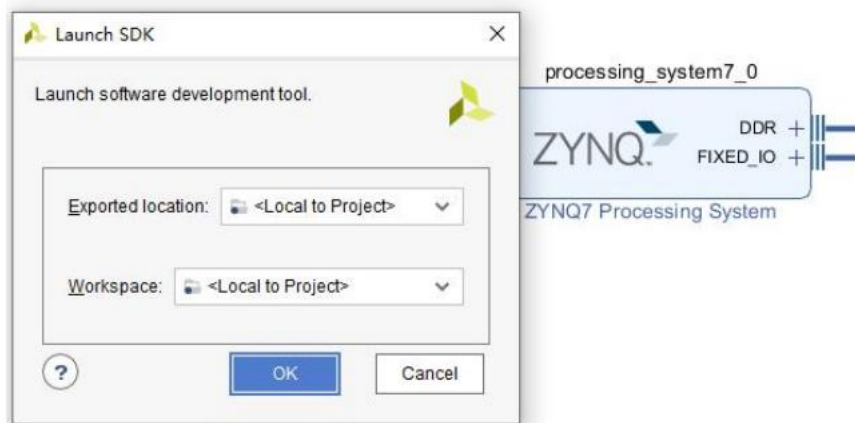
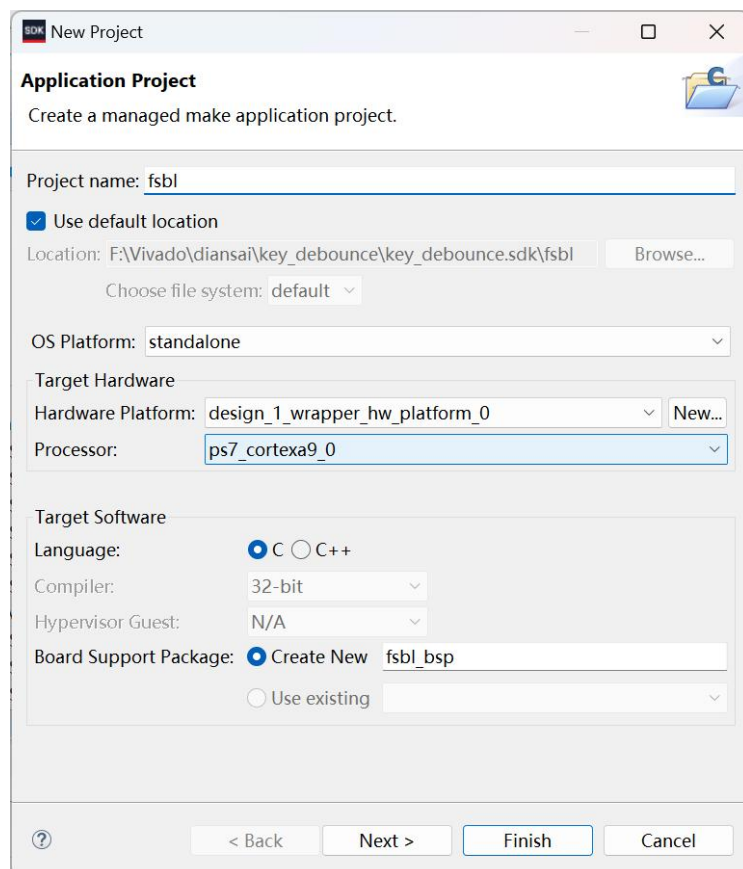
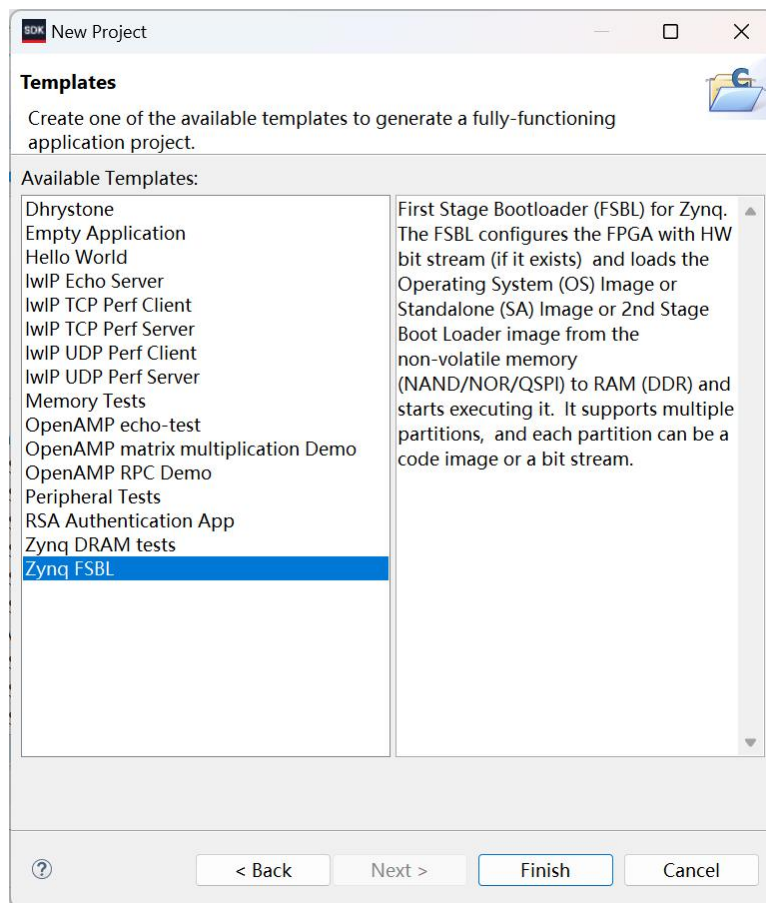


图 1.2.32 选择硬件路径及工作空间

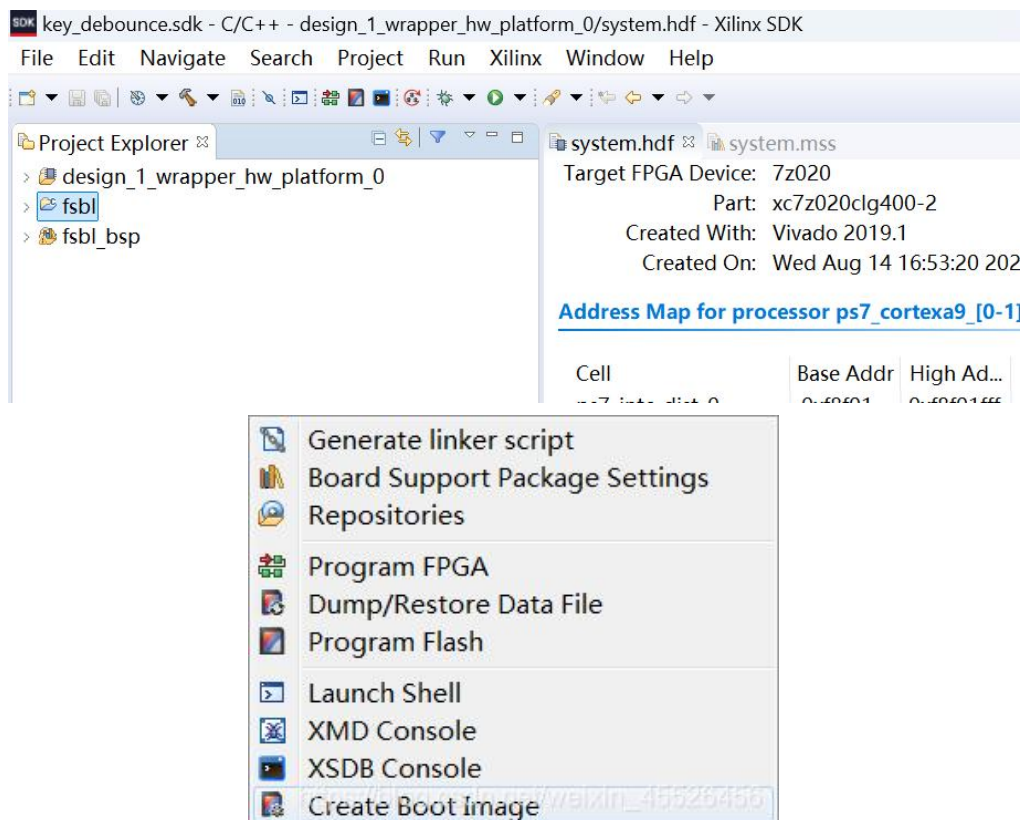
点击菜单 File -> New -> Application Project，新建一个名为 fsbl 的 APP，会出现 1 个 hardware platform: design_1_wrapper_hw_platform_0，大概选最新最上面那个就好（图上的 platform 可能不一致）。



点击 Next，选择 zynq fsbl，如下图所示。



右键 fsbl，然后选择 create boot image



下方方框出现两个文件即正常，bootloader 在上，bit 文件在下，可能有延迟稍等或请重试。

Basic Security

Output BIF file path: D:\vvdcode\led_guhua\led_guhua.sdk\fsbl\bootimage\fsbl.bif Browse...

UDF data: Browse...

☐ Split Output format: BIN

Output path: D:\vvdcode\led_guhua\led_guhua.sdk\fsbl\bootimage\BOOT.bin Browse...

Boot image partitions

File path	Encrypted	Authenti...
(bootloader) D:\vvdcode\led_guhua\led_guh...	none	none
D:\vvdcode\led_guhua\led_guhua.sdk\ledBD...	none	none

https://blog.csdn.net/weixin_45526456 Add

点击 creat image。

2.写入 flash

点击 xilinx-program flash。

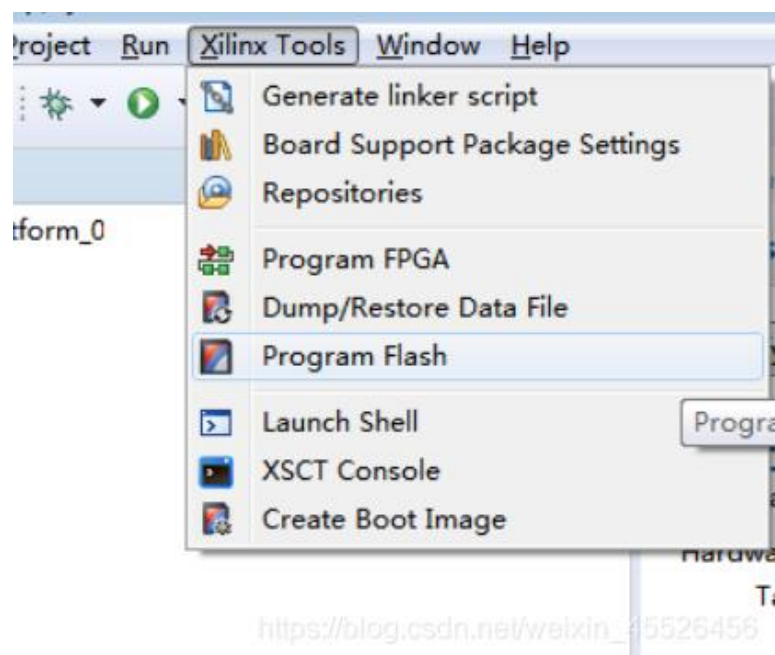
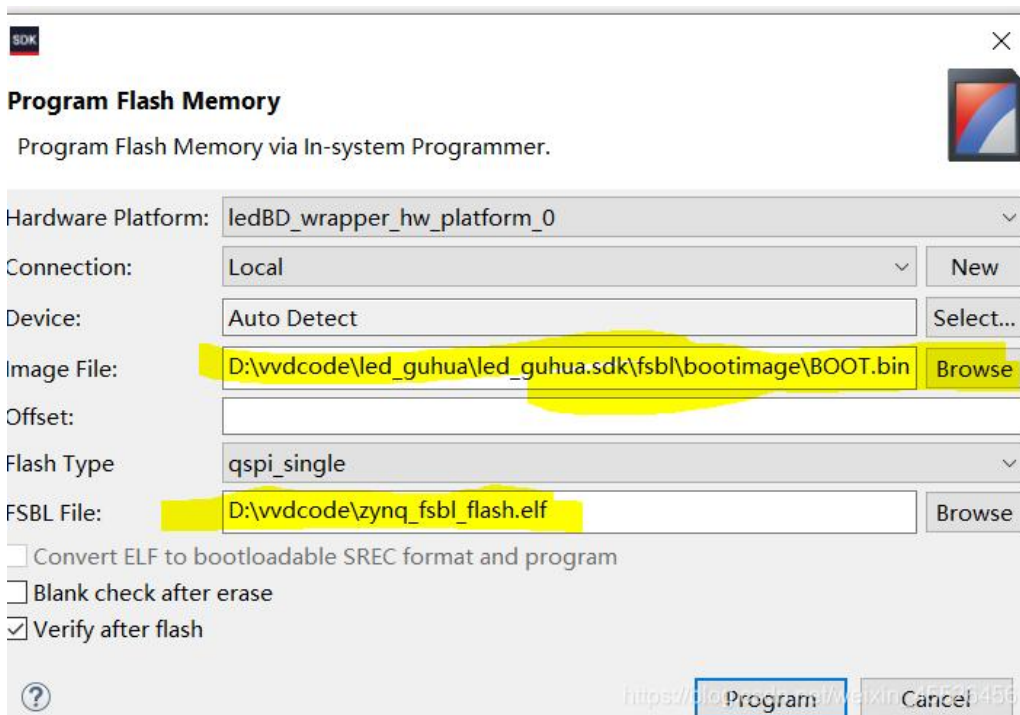
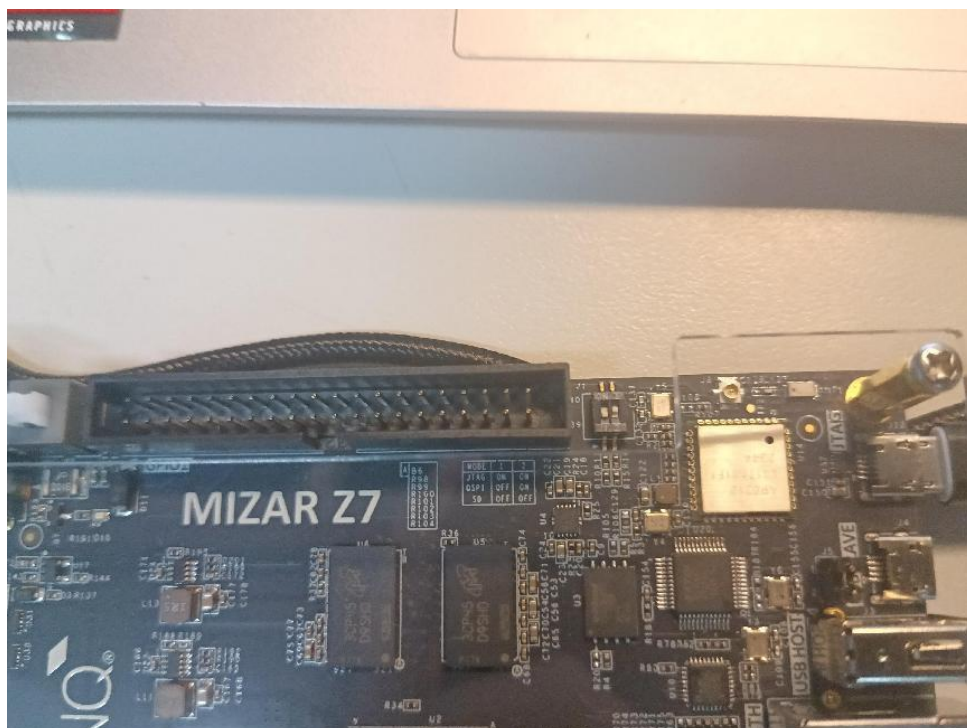


image file 选择上一步生成的 bin 文件！fsbl file 使用自己生成的 fsbl 即可，如下图所示。



等待烧写完成后，设置启动模式为 QSPI（不同厂商可能不同，我这里的是调整拨码开关位置来实现的，1—OFF，2—ON），关闭电源开关，再次启动，即可实现上电之后 PL 侧按键控制 LED 灯的功能。



最终结果：

